

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Constraint-Based Problem Solving

Course Code:	CSA0609	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 1 – Constraint Based Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.		
Student Name:	NANUBALA MADHUSUDHAN	Reg. No.:	192425303
Section / Batch:	Slot-C	Date:	27/08/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Apply backtracking techniques systematically for combinatorial problem solving.
- Clearly classify problems under P, NP, NP-Hard, and NP-Complete categories wherever required.
- Provide proper justification, state-space representation, and complexity analysis for all solutions.
- Neat presentation and logical explanation are mandatory.

Question Type	Focus Area	Questions
Constraint Based Problem Solving	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

SECTION A – CONSTRAINT-BASED PROBLEM SOLVING

Q22. Exact Cover Problem (Constraint Satisfaction / NP-Complete Problem)

1. Problem Statement

Given a universal set U of elements and a collection S of subsets of U , determine whether there exists a sub-collection of S — called an exact cover — such that every element of U is contained in exactly one of the chosen subsets. Each element must be covered, and no element may be covered more than once.

Input

- Universal set U of elements to be covered
- Collection $S = \{S_1, S_2, \dots, S_n\}$ of subsets of U

Output

- A sub-collection of S whose union equals U , with no two chosen subsets overlapping (an exact cover), if one exists
- Report of "no exact cover exists" if no such sub-collection can be found

2. Constraints

A subset can be selected only if the following conditions hold:

- Coverage constraint: Every element of U must eventually be included in at least one selected subset — no element may be left uncovered.
- Exclusivity constraint: No element may appear in more than one selected subset — selected subsets must be pairwise disjoint.
- Together, these two constraints mean each element of U is covered exactly once — neither zero times nor more than once.

For example, if subset $S_1 = \{1, 4\}$ is selected, no other subset containing 1 or 4 (such as $S_4 = \{3, 4, 5\}$) may also be selected.

3. Effect of Constraints

The exclusivity constraint sharply reduces the number of feasible combinations, because selecting one subset immediately removes every other subset that shares any element with it from further consideration. This interdependency between choices is what makes Exact Cover a genuine constraint satisfaction problem rather than a simple selection problem.

Example universe and subset collection:

Subset	Elements
S1	{1, 4}
S2	{1, 2}
S3	{2, 3}
S4	{3, 4, 5}
S5	{5}
S6	{2, 5}

Universe: $U = \{1, 2, 3, 4, 5\}$

If $S_1 = \{1, 4\}$ is selected, then S_2 (contains 1) and S_4 (contains 4) are immediately eliminated from consideration, since selecting either would cover element 1 or 4 twice. This is the exclusivity constraint actively pruning the search space. The algorithm must therefore explore alternative combinations to find a valid exact cover, if one exists.

4. Backtracking Algorithm

```
EXACT-COVER(U, S)
Input:
  U = universal set of elements
  S = collection of subsets of U
Output:
  An exact cover of U, or "no solution" if none exists

1. selected <- empty set
2. found <- BACKTRACK(U, S, selected)
3. If found = true, Return selected
4. Else Return "no exact cover exists"
```

BACKTRACK(remaining, S, selected)

```
1. If remaining = empty then
  Return true          // all elements covered exactly once
2. Pick element e with the fewest covering subsets in remaining
   (this is the constraint used for branching)
3. For each subset  $S_i$  in S that contains e:
  If  $S_i$  is disjoint from (U - remaining) then // exclusivity check
    Add  $S_i$  to selected
    remaining' <- remaining -  $S_i$ 
    If BACKTRACK(remaining', S, selected) = true then
      Return true
    Remove  $S_i$  from selected // undo / backtrack
4. Return false // no subset containing e leads to a solution
```

Choosing the least-covered element e at each step (rather than an arbitrary element) is itself a constraint-driven heuristic: it ensures the algorithm branches on the most constrained part of the problem first, which tends to detect infeasibility earlier and reduces wasted search.

5. Pruning

Pruning eliminates branches that cannot possibly lead to a valid exact cover. Two pruning rules apply directly from the constraints:

- **Overlap pruning:** If a candidate subset S_i shares any element with the subsets already selected, it is discarded immediately — it can never satisfy the exclusivity constraint on this branch.
- **Dead-element pruning:** If any remaining element has zero subsets left that (a) contain it and (b) do not overlap the current selection, the branch is abandoned immediately, since that element can never be covered.

```
If remaining element e has no valid non-overlapping subset containing it then
  Return false // this branch cannot cover e – prune
```

This means that as soon as any single element becomes impossible to cover under the current partial selection, the algorithm abandons that branch immediately rather than continuing to explore it — since no completion of that branch could ever satisfy the coverage constraint.

6. Worked Example

Using $U = \{1,2,3,4,5\}$ and the subsets above, the backtracking search proceeds as follows:

- Element 5 has the fewest covering subsets (S_4, S_5, S_6) — branch on it first
- Try $S_5 = \{5\}$: remaining = $\{1,2,3,4\}$. Try $S_1 = \{1,4\}$: remaining = $\{2,3\}$. Try $S_3 = \{2,3\}$: remaining = $\{\}$ → exact cover found: $\{S_1, S_3, S_5\}$
- Backtracking further would also find $\{S_2, S_4\} = \{1,2\} \cup \{3,4,5\}$ as a second valid exact cover
- Any branch that tries $S_6 = \{2,5\}$ together with $S_4 = \{3,4,5\}$ is pruned immediately, since both contain element 5 — an overlap violation

Both $\{S_1, S_3, S_5\}$ and $\{S_2, S_4\}$ are valid exact covers of U ; the algorithm returns the first one it finds and can be modified to enumerate all solutions if required.

7. Correctness

The algorithm is correct because at every step it:

1. Selects an uncovered element e and considers every subset that could cover it — no valid option is skipped.
2. For each candidate subset, verifies the exclusivity constraint (no overlap with subsets already chosen) before committing to it.
3. Recurses on the strictly smaller remaining set, guaranteeing termination since remaining shrinks each time a subset is chosen.
4. Undoes (backtracks) any selection that does not lead to a complete cover, so every combination of valid, non-overlapping subsets is implicitly explored.

Because the algorithm branches on every subset covering the chosen element and backtracks on failure, it explores the full space of valid selections. If any exact cover exists, some sequence of choices in this search tree corresponds to it, so the algorithm is guaranteed to find one. Pruning does not affect correctness because it only discards branches that have already violated a constraint (overlap) or can no longer satisfy one (an uncoverable element) — such branches could never have produced a valid solution.

8. Complexity

For a universe with $|U| = n$ elements and a collection of $|S| = m$ subsets, in the worst case each subset may or may not be selected, giving a search space of:

$$O(2^m)$$

In practice, the exclusivity pruning and the least-covered-element heuristic substantially reduce the number of branches actually explored, but the worst-case time complexity remains exponential, since Exact Cover is NP-Complete.

Space complexity is approximately:

$$O(n + m)$$

for the recursion stack, the current selection, and the tracking of remaining uncovered elements.

9. Complexity Classification

- Exact Cover is in NP: given a proposed sub-collection of subsets, it can be verified in polynomial time that the subsets are pairwise disjoint and their union equals U.
- Exact Cover is NP-Hard: it is one of Karp's original 21 NP-Complete problems, and well-known problems such as Sudoku, N-Queens variants, and tiling puzzles can be reduced to it in polynomial time.
- Since Exact Cover is both in NP and NP-Hard, it is classified as NP-Complete.
- No polynomial-time algorithm is known for Exact Cover in general; the best known exact algorithms (such as Knuth's Algorithm X with dancing links) are still exponential in the worst case, though efficient in practice for many structured instances.

10. Conclusion

Backtracking finds a valid exact cover by exploring subset selections that satisfy both the coverage and exclusivity constraints, undoing any choice that leads to a conflict or an uncoverable element. Pruning — through overlap checks and dead-element detection — removes large parts of the search space early, using the most-constrained-element heuristic to fail fast on infeasible branches. Although this backtracking approach has exponential worst-case complexity, this is expected and unavoidable in general, since the Exact Cover problem is NP-Complete: no known algorithm solves it in polynomial time for all instances, and any solution correctness therefore relies on a systematic, prune-aware search of the kind developed here.

Code of Conduct

I, NANUBALA MADHUSUDHAN (192425303), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: NANUBALA MADHUSUDHAN

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Marks Evaluation

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%				
Constraint Analysis	25%				
Solution Development	25%				
Application of Concepts	15%				
Justification	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____